



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
SAGARMATHA ENGINEERING COLLEGE**

**A
PROJECT PROPOSAL REPORT
ON
QSAFE NEPAL: A MULTILINGUAL EARTHQUAKE RESPONSE
ASSISTANT USING RETRIEVAL-AUGMENTED GENERATION**

SUBMITTED BY

**KAPIL NATH SEC080BCT018
PRASUN JOSHI SEC080BCT030
RITESH THAKUR SEC080BCT032
SHIVAM SINGH SEC080BCT039**

**A PROPOSAL REPORT SUBMITTED TO THE DEPARTMENT OF
ELECTRONICS AND COMPUTER ENGINEERING IN PARTIAL
FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR IN COMPUTER/ELECTRONICS ENGINEERING**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
SANEPA, LALITPUR, NEPAL**

June, 2026

ABSTRACT

Nepal sits right on the collision zone of the Indian and Eurasian tectonic plates, making it one of the most earthquake-prone places on Earth. The devastating 2015 Gorkha earthquake (M_w 7.8)—which tragically claimed over 8,900 lives, injured 22,300 people, and displaced 2.8 million citizens—made one thing terrifyingly clear: when a disaster hits, a lack of immediate, reliable information costs lives. Unfortunately, current disaster apps stop working when the internet goes down, rely on English or formal Nepali that many locals struggle to understand, and standard AI chatbots can easily hallucinate unsafe advice. To solve this, we developed QSAFE Nepal, a mobile-friendly Progressive Web Application (PWA) designed to deliver bilingual earthquake safety info you can actually trust. By using Retrieval-Augmented Generation (RAG), the app’s AI assistant strictly pulls its answers from verified safety documents published by Nepal’s National Disaster Risk Reduction and Management Authority (NDRRMA). We integrated NLP.js so users can chat naturally in either English or casual Nepali, and built the app with an offline-first design using service workers and local SQLite storage. This ensures that even when cell towers are completely down, anyone can still access emergency contacts, survival instructions, and damage-reporting tools. On top of that, the system includes a predictive machine learning script that hooks into live United States Geological Survey (USGS) seismic data to simulate and map out real-time aftershock patterns across major Nepali cities as they happen. The entire platform balances a smart, bilingual AI assistant with a completely offline emergency directory. By relying on open-source tools, free APIs, and our own laptops, we’ve made sure this project is highly practical, budget-friendly, and fully achievable within the timeframe of an eight-week undergraduate minor project.

Keywords: Earthquake Safety, Retrieval-Augmented Generation, Nepali NLP, Offline-First Architecture, Disaster Response, Progressive Web Application

Contents

- ABSTRACT i**
- LIST OF FIGURES v**
- LIST OF TABLES vi**
- LIST OF ABBREVIATIONS 1**
- 1 INTRODUCTION 2**
 - 1.1 Background 2
 - 1.2 Problem Statement 3
 - 1.3 Objectives 3
 - 1.4 Features 3
 - 1.5 Feasibility 4
 - 1.5.1 Technical Feasibility 4
 - 1.5.2 Operational Feasibility 4
 - 1.6 System Requirements 5
 - 1.6.1 Software Requirements 5
 - 1.6.2 Hardware Requirements 5
 - 1.6.3 Network Requirements 5
 - 1.6.4 Functional Requirements 5
 - 1.6.5 Non-Functional Requirements 6
- 2 LITERATURE REVIEW 7**
- 3 RELATED THEORY 9**
 - 3.1 Facts 9
 - 3.2 Retrieval-Augmented Generation (RAG) 9
 - 3.2.1 Data Ingestion Phase 9
 - 3.2.2 Online Retrieval and Generation Phase 10
 - 3.3 Natural Language Processing and Intent Classification 11
 - 3.4 Vector Databases and Semantic Search 11
 - 3.5 Offline-First Architecture and Progressive Web Applications 12

3.5.1	Service Workers	12
3.5.2	Local Data Persistence	12
3.6	Text Embeddings and the Google Embedding API	13
3.7	Introduction to Web Technologies	13
3.7.1	HyperText Markup Language (HTML)	13
3.7.2	Cascading Style Sheets (CSS)	13
3.7.3	JavaScript (JS)	14
3.8	Application Programming Interface (API)	14
3.9	Frameworks	14
3.9.1	NLP.js	15
3.9.2	Node.js and Express.js	15
3.9.3	FastAPI	15
3.9.4	Node-Cron and Axios:	15
3.10	Libraries	16
3.10.1	Client-Side and Offline-First Libraries	16
3.10.2	Backend, RAG, and Orchestration Libraries	16
3.10.3	Core Utility Libraries	17
3.11	Performance Metrics	17
3.11.1	Accuracy	17
3.11.2	Precision	17
3.11.3	Recall	18
3.11.4	F1-Score	18
3.11.5	Confidence Threshold (C_t)	18
4	METHODOLOGY	19
4.1	System Block Diagram	19
4.1.1	Input Section	20
4.1.2	Processing Section	20
4.1.3	Output Section	21
4.2	Dataset	22
4.2.1	Nature of the Datasets	23
4.2.2	Properties of the Datasets	23
4.3	Preparation of Datasets	24
4.3.1	Dataset Statistics	25
4.4	Algorithm	26
4.5	Flowchart	28
4.6	Software Development Lifecycles	29
4.7	Entity–Relationship (ER) Diagram	30
4.8	Use Case Diagram	32

5	EXPECTED OUTPUT	33
A	APPENDIX	36
A.1	Gantt Chart / Project Timeline	36

List of Figures

3.1	A general architecture of RAG Pipeline [13]	10
3.2	Block diagram of Vector Database [15]	12
4.1	System Block Diagram of the Proposed multilingual earthquake management system with Explainability	19
4.2	Nepal seismicity master.csv Dataset File [14]	23
4.3	Flowchart of the A MULTILINGUAL EARTHQUAKE RESPONSE ASSISTANT with Explainability	28
4.4	Agile Model Workflow with Feedback Loop	29
4.5	Entity–Relationship (ER) Diagram Of QSAFE Nepal	30
4.6	Use Case Diagram Of QSAFE Nepal	32
A.1	Gantt Chart for QSAFE Nepal. Project Timeline (June–August 2026)	36

List of Tables

1.1	Technical Feasibility Assessment	4
1.2	Software Requirements	5
1.3	Hardware Requirements	5
4.1	Nature of Datasets Used	23
4.2	Properties of NDRRMA Manual Chunks	24
4.3	Properties of Intent Training Corpus	24
4.4	Properties of Local Client Damage Reports	24
4.5	Class Distribution of Intent Dataset	26
4.6	Distribution of Knowledge Base Document Chunks	26

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GPU	Graphics Processing Unit
LLM	Large Language Model
NDRRMA	National Disaster Risk Reduction and Management Authority
NLP	Natural Language Processing
PWA	Progressive Web Application
RAG	Retrieval-Augmented Generation
SDLC	Software Development Life Cycle
SQLite	Structured Query Language Lite (Lightweight Embedded Database)
SUS	System Usability Scale
UI	User Interface
URL	Uniform Resource Locator
USGS	United States Geological Survey
UTF-8	Unicode Transformation Format – 8-bit

CHAPTER 1

INTRODUCTION

1.1 Background

Nepal occupies a geographically vulnerable position along the Himalayan seismic belt, where the northward movement of the Indian Plate beneath the Eurasian Plate builds tectonic stress that releases periodically as destructive earthquakes. The National Seismological Centre of Nepal records hundreds of minor tremors annually, and major events occur with enough regularity that earthquake preparedness is a recognized public concern. The 2015 Gorkha earthquake (Mw 7.8) resulted in 8,964 confirmed fatalities, over 22,300 injuries, the destruction of more than 600,000 houses, and the displacement of nearly 2.8 million people across 31 districts [1].

Disaster researchers note a critical period immediately following a major seismic event during which accurate safety information can help reduce secondary casualties. Secondary casualties arise not from the earthquake itself but from subsequent decisions made on incorrect information, such as re-entering structurally compromised buildings, improper handling of gas leaks, or ignoring aftershock warnings. These preventable incidents tend to affect rural communities more severely, where access to formal information channels is limited.

The National Disaster Risk Reduction and Management Authority (NDRRMA) publishes earthquake safety manuals containing detailed protocols for preparedness and response. Within the system architecture, the USGS API serves as the primary real-time data streaming layer, providing live seismic telemetry that allows the backend to dynamically verify local tremors and cross-reference crowdsourced user reports. However, these materials exist primarily as static PDF documents, accessible mainly to those with internet connectivity and sufficient literacy to navigate them. The broader digital landscape for disaster response in Nepal is further constrained by language barriers, internet dependency, and the absence of interactive query systems. Recent developments in natural language processing, lightweight AI tools, and Progressive Web Application (PWA) technologies now make it practical for small student teams to build systems that can partially address these limitations at low cost. This project proposes QSAFE Nepal, a system built for Nepal's seismic and linguistic context, combining document-grounded information retrieval with bilingual query support and partial offline functionality [9].

1.2 Problem Statement

Despite the availability of earthquake safety information in Nepal, three practical gaps prevent this information from reaching citizens when and where it is most needed.

Language and Literacy Barriers: Official disaster response materials in Nepal are predominantly published in English or formal standard Nepali. Approximately 44.6 percentage of Nepal's population speaks Nepali as a first language, while the remainder speaks one of over 120 other languages and dialects . No currently deployed disaster response application provides intent-based query support in simplified Nepali suitable for users with low literacy.

Unverified AI Responses in Safety-Critical Contexts: General-purpose large language model assistants may produce fluent responses that do not accurately reflect specific safety procedures . In a life-critical scenario, a response that contradicts official NDRRMA protocols could cause harm. Restricting AI responses to verified source documents reduces this risk.

Internet Connectivity Dependency: Nepal's telecommunications infrastructure can be significantly disrupted during and after major seismic events. Applications that depend entirely on cloud-based services may become unavailable precisely when they are most needed.

1.3 Objectives

The objectives of this project are:

- To develop an adaptive, offline-first assistant that provides cross-lingual, multilingual support when online, while guaranteeing a robust bilingual (English/Nepali) fallback system when completely offline.
- To develop an offline-first PWA for emergency contacts, safety guidelines, and damage reporting, while integrating live USGS API telemetry to verify real-time seismic events when online.

1.4 Features

The features of our project are as follows:

- **A Bilingual Safety Bot:** A chat interface that answers safety queries in English and Nepali using only verified data from NDRRMA manuals.
- **Damage Reporter:** A form for users to submit structural damage reports (e.g., cracks, gas leaks) which are stored locally and synced when online.

- **Offline Emergency Directory:** A preloaded list of emergency contacts (100, 102, 16666) and shelter locations accessible without internet.
- **User-Friendly Interface:** Offers an intuitive platform where users can input emergency queries and instantly receive verified earthquake safety responses.
- **Live Earthquake Verifier:** A real-time tracking feature that connects to the USGS API to detect active tremors instantly, helping the app verify ongoing emergencies and spot local crises based on sudden spikes in user searches.

1.5 Feasibility

1.5.1 Technical Feasibility

All technologies proposed in this project are open-source or available via free-tier APIs, actively maintained, and well-documented. React.js and Node.js are familiar to the team from coursework. NLP.js supports Nepali among its supported languages and runs within Node.js without requiring GPU resources. ChromaDB is deployable on standard hardware with a JavaScript API. Google’s text-embedding-004 and Gemini 2.0 Flash models are accessible via the Gemini free-tier API without requiring local GPU hardware, making them suitable for the team’s available setup.

Table 1.1: Technical Feasibility Assessment

Technology	Availability	Maturity	Team Familiarity
React.js + Vite	Open source	High	High
Node.js	Open source	High	High
NLP.js	Open source	Medium	Medium
ChromaDB	Open source	Medium	Low
text-embedding-004	Google API	High	Low
Gemini 2.0 Flash	Google API	High	Low
SQLite (sql.js)	Open source	High	Medium

1.5.2 Operational Feasibility

The system is delivered as a Progressive Web Application accessible from any modern smartphone browser without requiring an app store installation. The interface supports both free-text input and pre-set query buttons to accommodate users with limited typing experience. Nepali language support uses Unicode UTF-8, which is compatible with modern devices.

1.6 System Requirements

The system requirements of our project are:

1.6.1 Software Requirements

The software requirements for our project are as follows:

Table 1.2: Software Requirements

Category	Software / Tool	Version
Operating System	Windows 10/11, Ubuntu 20.04+	Latest stable
Frontend Framework	React.js with Vite	v18+
Backend Runtime	Node.js	v18 LTS
NLP Library	node-nlp (NLP.js)	v4+
Vector Store	ChromaDB	Latest
Embedding Model	text-embedding-004 (Google)	Latest
LLM	Google Gemini 2.0 Flash (API)	Latest
Database	SQLite (sql.js)	Latest
Version Control	Git + GitHub	Latest
IDE	Visual Studio Code	Latest

1.6.2 Hardware Requirements

The hardware requirements for our project are as follows:

Table 1.3: Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or higher
Storage	50 GB HDD/SSD	100 GB SSD
Network	Stable broadband (dev)	10 Mbps broadband
End-User Device	Any smartphone (Chrome 90+)	Android 8+ or iOS 13+

1.6.3 Network Requirements

- Stable connection for downloading pre-trained models and datasets
- Minimum bandwidth: 15 Mbps for standard access

1.6.4 Functional Requirements

The system's core capabilities are broken down into the following functional requirements:

- (a) Multilingual Input: The system shall accept user queries in English and Nepali (Unicode UTF-8).
- (b) Verified Response: The system shall generate safety responses only from retrieved NDRRMA document content.
- (c) Confidence Thresholding: If the similarity score of the top retrieved chunk falls below a defined threshold, the system shall return a fallback message rather than an unverified response.
- (d) Offline Directory Access: The system shall provide emergency contacts and shelter locations from cached local data without requiring internet.
- (e) Damage Reporting: The system shall allow users to store damage reports locally and synchronize them when connectivity is restored.
- (f) Language Detection: The system shall automatically detect English or Nepali input without requiring manual language selection by the user.
- (g) Source Attribution: Every safety response shall reference the specific NDRRMA section from which it was retrieved.

1.6.5 Non-Functional Requirements

- (a) Performance: The system should provide responses within a reasonable time under normal operating conditions. Offline directory access should not require server communication.
- (b) Reliability: The system should maintain normal availability under standard conditions and degrade gracefully to offline mode when connectivity is lost.
- (c) Usability: The interface shall be evaluated using the System Usability Scale (SUS) questionnaire with a small group of test participants to assess whether the system is accessible to its intended users.
- (d) Security: Locally stored damage reports shall not include personally identifiable information. No mandatory login shall be required for damage report submission.
- (e) Portability: The system shall function as a PWA installable from any modern mobile browser without requiring an app store installation.

CHAPTER 2

LITERATURE REVIEW

The development of QSAFE Nepal draws on four active research areas: AI-assisted disaster response, retrieval-augmented generation, multilingual natural language processing for low-resource languages, and offline-first application architecture. This chapter reviews five works that together establish the context and technical motivation for the proposed system, identifies the limitations of each, and explains how QSAFE Nepal attempts to address the resulting gaps.

Introduced the RAG framework to augment language models with an external document index, significantly improving factual accuracy over purely generative models. This anchors responses to verifiable documents rather than learned patterns. While original RAG frameworks depend heavily on embedding quality across vast collections, QSAFE Nepal optimizes this by narrowing the knowledge base to a single authoritative source: the NDRRMA manual. [7]

Categorized AI hallucinations into intrinsic (contradicting source text) and extrinsic (introducing unverified data) types, noting higher risks in low-resource domains like Nepal-specific disaster response. The authors recommended closed-world generation constraints to mitigate this. QSAFE Nepal implements this strategy through its restricted RAG pipeline and similarity-based confidence thresholding, while addressing the original paper’s gap regarding English–Nepali bilingual configurations. [5]

Classified world languages by resources, identifying Nepali as a mid-tier, relatively lower-resource language requiring careful tool selection for domain-specific tasks. This motivates the project’s selection of NLP.js over massive transformer alternatives. For a constrained four-intent classification task, NLP.js provides predictable behavior, operates without expensive GPU resources, and is highly trainable on small datasets. [8] [5]

Documented the breakdown of internet-dependent disaster tools during the 2010 Haiti and 2015 Nepal earthquakes, establishing that offline functionality must be a primary design requirement rather than an optional add-on. QSAFE Nepal applies this offline-first principle by leveraging Progressive Web Application (PWA) service worker caching alongside the sql.js WebAssembly SQLite runtime. [3]

Evaluated existing disaster chatbots across accuracy, language accessibility, and network resilience, concluding that no system performed well in all three simultaneously. Curated

knowledge retrieval, multi-language processing, and offline operation have rarely been combined. QSAFE Nepal is engineered specifically to address these three dimensions concurrently within a unified framework. [6]

Overall, the five works reviewed above support three propositions that motivate QSAFE Nepal. First, RAG architecture is a practical method for grounding AI responses in verified source material for domain-specific applications. Second, lightweight NLP tools such as NLP.js are more appropriate than large transformer models for constrained intent classification in resource-limited deployment settings. Third, offline-first architecture is a design requirement rather than an optional feature for disaster response tools intended to remain functional during the events they are meant to address. Existing systems reviewed in these studies address at most two of these three requirements; QSAFE Nepal is designed to address all three within a realistic undergraduate project scope.

CHAPTER 3

RELATED THEORY

3.1 Facts

The devastating impact of earthquakes, like the 2015 disaster in Nepal, shows just how badly we need immediate, reliable safety advice when a crisis hits. Right now, most emergency tools depend entirely on a steady internet connection or massive cellular networks. But when a major earthquake strikes, cell towers go down and networks crash—leaving people completely cut off from the help they need. On top of that, official safety manuals are often written only in English, creating a huge barrier for rural communities who need that information in their native Nepali.

While modern AI tools like ChatGPT can answer safety questions, they require a lot of internet data, need expensive hardware, and can sometimes make up incorrect information. QSAFE Nepal fixes these problems by building an app that works completely offline. It combines a fast, lightweight local search system with an internet-independent backup database. By locking the app’s knowledge strictly to official guidelines from the National Disaster Risk Reduction and Management Authority (NDRRMA), it gives people accurate, lifesaving answers in both English and Nepali; even when there is absolutely no cell service or internet available.

3.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation is an architectural pattern that improves the factual accuracy of Large Language Model (LLM) outputs by providing the model with relevant retrieved text at inference . Unlike a standard LLM, which generates responses solely from patterns learned during training, a RAG system dynamically retrieves context from an external knowledge base for each user query. This retrieved content is included in the prompt given to the LLM, which is then instructed to base its response on the provided material rather than on general training knowledge.[7]. The RAG pipeline in QSAFE Nepal operates in two phases.

3.2.1 Data Ingestion Phase

The NDRRMA Earthquake Safety Manual is loaded and its text is extracted. The extracted text is divided into overlapping chunks so that related content at section boundaries is not lost.

Each chunk is converted into a dense vector embedding using Google’s text-embedding-004 model via the Gemini API. These embeddings and their associated text chunks are stored in ChromaDB along with metadata recording the source document, page number, and section title.

3.2.2 Online Retrieval and Generation Phase

When a user submits a query, it is converted to a vector embedding using the same model. A similarity search is then performed against the ChromaDB index to find the most semantically similar document chunks. If the similarity score of the top result meets a defined confidence threshold, the retrieved chunks are passed as context to the LLM, which generates a response using only the provided context. If the similarity score falls below the threshold, a fallback message is returned without LLM generation, informing the user that a verified answer was not found for the query. The key property that makes RAG suitable for safety-critical applications is traceability: every response is linked to a specific retrieved document chunk, which can be verified against the original source. Figure 3.1 illustrates the general RAG pipeline architecture followed in this project.

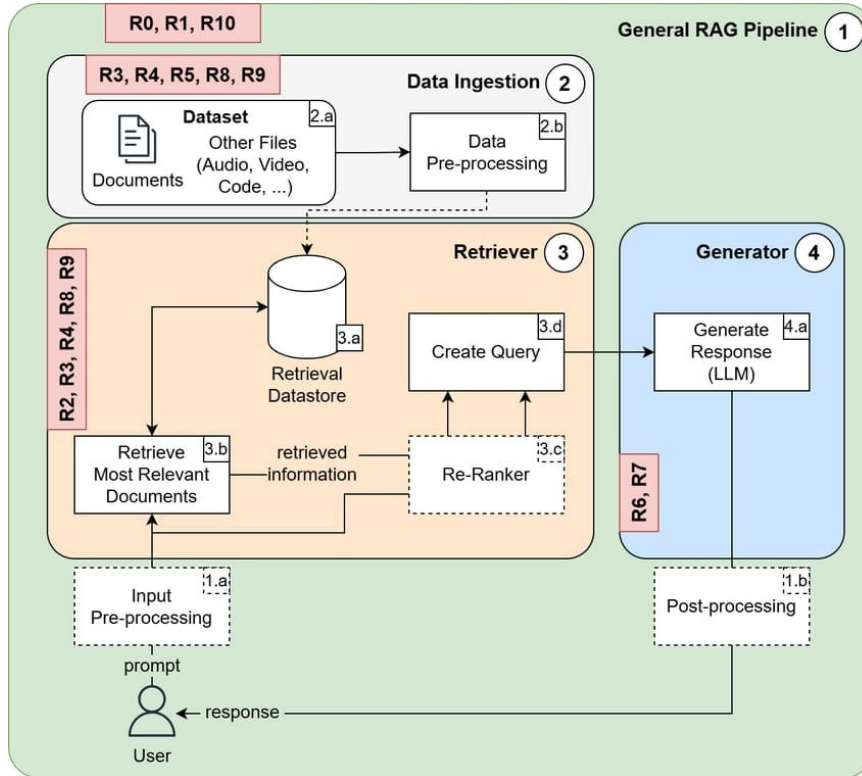


Figure 3.1: A general architecture of RAG Pipeline [13]

3.3 Natural Language Processing and Intent Classification

Natural Language Processing (NLP) encompasses computational methods for processing and interpreting human language. In QSAFE Nepal, the NLP.js library serves three functions. Language Detection determines whether the user’s input is in English or Nepali. NLP.js applies character n-gram frequency profiles for this purpose. Nepali’s use of the Devanagari script (Unicode range U+0900–U+097F) provides an additional reliable signal for script-based detection. Intent Classification assigns the user’s query to one of a predefined set of intent categories using a combination of a Naïve Bayes classifier and pattern matching, trained on domain-specific example sentences. The two primary intents defined for this system are: Safety Query and Emergency/Damage Request. Entity Extraction identifies specific named entities within queries, such as location or damage type (for example, “structural crack” or the Nepali equivalent), which are used to route damage reports appropriately. NLP.js was chosen over transformer-based alternatives such as mBERT or XLM-R for practical reasons: it requires no GPU, runs within Node.js without a separate microservice, and is sufficient for a two-intent classification task within the scope of this project Joshi et al. [2020].

3.4 Vector Databases and Semantic Search

A vector database stores high-dimensional numerical vector representations of text and supports efficient similarity search over them. In QSAFE Nepal, ChromaDB stores the vector embeddings generated from NDRRMA document chunks. When a user query is received, it is embedded using the same model used during ingestion. The semantic similarity between the query vector and each stored chunk vector is measured using cosine similarity, defined as:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \quad (3.1)$$

where $\mathbf{A}$ is the query vector and $\mathbf{B}$ is a stored document chunk vector. A similarity value of 1.0 indicates identical semantic content; values approaching 0 indicate unrelated content. ChromaDB was selected because it is open-source, persists on local disk without a cloud subscription, and provides a JavaScript client compatible with the Node.js backend.

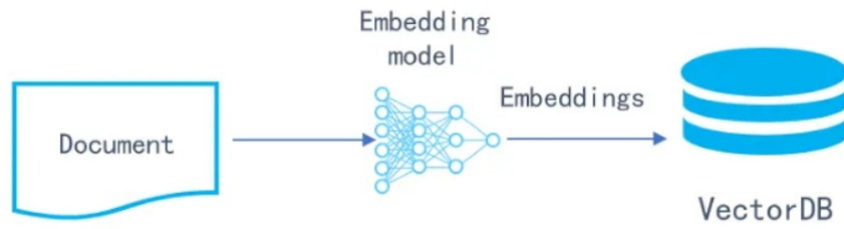


Figure 3.2: Block diagram of Vector Database [15]

3.5 Offline-First Architecture and Progressive Web Applications

An offline-first application is designed so that core functionality works without network connectivity, treating network access as a supplementary rather than mandatory condition Meier [2015]. This is implemented in QSAFE Nepal through two mechanisms:

3.5.1 Service Workers

A service worker is a JavaScript script that runs in the browser background, intercepting outgoing network requests. Static assets and pre-loaded emergency data are cached during the first application load and served from cache on subsequent visits regardless of connectivity status.

3.5.2 Local Data Persistence

User-generated damage reports are stored in a SQLite database running in the browser via WebAssembly (sql.js). When internet connectivity is detected, the application attempts to synchronize pending reports to the server. The system is designed to operate in three connectivity states:

- Online: Full RAG pipeline is available; both modules are fully active.
- Degraded: Emergency directory and cached safety rules are available; the RAG pipeline is unavailable.
- Offline: Emergency directory and damage report storage are accessible; no server communication takes place.

A Progressive Web Application (PWA) is a web application that can be installed on a user's device from a browser without using an app store, and that supports service worker-based offline caching. Delivering QSAFE Nepal as a PWA allows users to access the system on

any modern smartphone browser without a dedicated app download, which is particularly relevant in low-bandwidth environments.

3.6 Text Embeddings and the Google Embedding API

A text embedding model converts text into a fixed-length dense vector such that semantically similar texts produce similar vectors Lewis et al. [2020]. In this project, Google’s text-embedding-004 model is used via the Gemini API to generate embeddings for both document chunks during ingestion and user queries during retrieval. The model encodes input text through multiple transformer layers, each applying self-attention to capture contextual relationships between words, producing a high-dimensional vector representation. This embedding setup is used only for searching through the NDRRMA text manual and the live data coming from the USGS API skips the AI embedding process entirely and is checked directly using geographic coordinates. The resulting embeddings are stored in ChromaDB during the ingestion phase and compared against query embeddings during retrieval using cosine similarity. The model was selected because it is accessible via a free-tier API without requiring local GPU resources, integrates naturally with the Node.js backend, and delivers strong multilingual performance that generalizes across English and Nepali text.

3.7 Introduction to Web Technologies

Web technologies are used to enable interaction between users and the fake news detection system. They provide a medium through which users can submit news text and receive classification results along with explanations. These technologies act as a bridge between the frontend interface and the backend transformer-based model.

3.7.1 HyperText Markup Language (HTML)

HTML is used to structure the web interface of the system. It defines the layout and components of the application, including text input fields, buttons, and result display sections. HTML ensures that the content is organized and accessible to users.

3.7.2 Cascading Style Sheets (CSS)

CSS is used to control the visual presentation of the web interface. It improves readability and user experience by defining layout design, colors, fonts, and spacing. CSS also helps make the interface responsive across different screen sizes.

3.7.3 JavaScript (JS)

JavaScript is used to add interactivity and dynamic behavior to the web application. It handles user input events, sends asynchronous requests to the backend API, and updates the user interface with classification results and explanations without reloading the page.

3.8 Application Programming Interface (API)

An Application Programming Interface (API) is basically the bridge that allows different software components to talk to each other. In our QSAFE Nepal system, the API connects the front-end React.js web app (PWA) with our backend cloud setup, which runs the Google Gemini 2.0 Flash model and the text-embedding-004 pipeline. When a user asks an earthquake safety question in English or Nepali, the API securely passes that query to the cloud and brings back a reliable, context-aware safety response in a clean JSON format.

Behind the scenes, the API handles the heavy lifting. It decodes incoming user requests, runs high-dimensional vector searches through ChromaDB, triggers the AI model, and formats the final answer. By separating our lightweight frontend from these resource-heavy AI tasks, the app stays fast, modular, and easy to scale. This separation also means we can update our safety manuals or fine-tune the AI backend whenever we want without breaking the offline-first app running on the user's phone.

On top of this internal AI manager, the system also connects directly to the external United States Geological Survey (USGS) API. This live data feed uses standard REST protocols to send real-time earthquake updates formatted in GeoJSON. While our internal API handles text and chatbot queries, the USGS API focuses entirely on raw seismic data like magnitudes, timestamps, and exact coordinates. Our backend automatically reads this live stream and filters it so we only track tremors happening within Nepal. Keeping this live tracking separate from the main chatbot database keeps the system highly efficient, prevents slowdowns when traffic spikes during emergencies, and ensures users get instant updates.

3.9 Frameworks

Frameworks provide structured, standardized environments that abstract low-level operations, simplifying the development, integration, and deployment of software systems. In the QSAFE Nepal project, a combination of client-side natural language parsing frameworks, scalable runtime environments, and rapid backend API frameworks are utilized to support the dual offline-first and online RAG pipeline.

3.9.1 NLP.js

NLP.js is an open-source, lightweight natural language processing framework designed specifically for Node.js environments. Unlike large transformer models that require significant computational resources and active internet access, NLP.js features a fast, rule-based and neural network classifier that can execute natively on the client side. In this project, NLP.js is compiled to run directly within the browser cache. It serves as the local intent classifier, analyzing user phrases offline to map them to specific emergency actions (such as identifying medical requests or safety queries) without sending data to a remote cloud server.

3.9.2 Node.js and Express.js

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser, making it highly efficient for building scalable network applications. Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. Within the QSAFE Nepal architecture, the backend server is constructed using Node.js and Express.js to manage the routing matrix of the hybrid cloud pipeline, handling inbound API requests, executing document retrieval sequences, and communicating with the vector search database.

3.9.3 FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It natively supports asynchronous programming, which allows it to handle concurrent requests with minimal latency. In the proposed system, FastAPI is deployed on the cloud-hybrid tier to expose the heavy linguistic components of the Retrieval-Augmented Generation (RAG) pipeline. It interfaces directly with python-centric machine learning libraries, orchestrating vector similarity computations and managing the prompt injection layer for the large language model.

3.9.4 Node-Cron and Axios:

This pair functions as your automated Earthquake Verifier. Node-cron runs a background clock that triggers Axios every few minutes to make a silent HTTP request to the external USGS servers. It fetches live GeoJSON tectonic data, brings it into your system, and applies the geographic boundaries to see if a tremor hit Nepal.

3.10 Libraries

The following specialized software libraries are integrated across the client application and the cloud-hybrid backend to support local data storage, vector search, and language processing pipelines.

3.10.1 Client-Side and Offline-First Libraries

- **sql.js:** A JavaScript library that uses WebAssembly to compile and run a full relational SQLite database directly inside the browser's memory. In QSAFE Nepal, it executes relational SQL queries against pre-cached emergency databases to retrieve local contact lists and safety assets entirely offline.
- **LocalForage:** An asynchronous browser storage library that enhances offline storage by wrapping IndexedDB or localStorage into a simple API. In QSAFE Nepal, it securely persists large application state files and offline database blobs directly onto the user's local device hardware storage.
- **Workbox:** A collection of libraries developed by Google to simplify the injection of caching and routing logic into Progressive Web Application Service Workers. In QSAFE Nepal, it handles background caching strategies for the user interface and language templates, ensuring the application loads instantly during network outages.

3.10.2 Backend, RAG, and Orchestration Libraries

- **LangChain:** An orchestration framework designed to simplify the development of applications powered by Large Language Models. In QSAFE Nepal, it automates the document ingestion pipeline by chunking the official NDRRMA safety manual text and feeding it into the model prompt context.
- **ChromaDB:** An open-source, lightweight vector database designed to store and perform low-latency semantic similarity searches over high-dimensional embeddings. In QSAFE Nepal, it hosts the vectorized safety manual segments and executes mathematical computations to match user questions with relevant context.
- **Transformers:** A comprehensive machine learning library that provides tools and pipelines to download and run pretrained deep learning models. In QSAFE Nepal, it converts raw user text into high-dimensional numerical vectors during the semantic similarity analysis step of the RAG pipeline.

3.10.3 Core Utility Libraries

- **Axios:** A Promise-based HTTP client library utilized to make asynchronous web requests from web applications to remote servers. In QSAFE Nepal, it transmits online user queries to the remote API and uses strict timeouts to trigger the local offline fallback if communication towers fail. The HTTP client your backend needs to send a request to the official USGS API endpoint (earthquake.usgs.gov) and download the live earthquake data (GeoJSON) into your app.
- **Dotenv:** A zero-dependency configuration module that loads sensitive variables from a hidden environment file into the system runtime. In QSAFE Nepal, it safeguards backend credentials, API keys, and vector database connection parameters across development and deployment environments.

3.11 Performance Metrics

Performance metrics provide objective, quantitative measures to evaluate the efficacy, reliability, and computational efficiency of a system. Unlike standard text classification architectures that rely solely on a single set of statistical model evaluations, the framework for QSAFE Nepal is split across distinct mathematical and information-retrieval models. This multi-tiered evaluation methodology guarantees a comprehensive assessment of the online RAG pipeline, the bilingual NLP intent classifier, and the client-side offline caching environment. These tracking parameters include Accuracy, Precision, Recall, and F1-Score.

3.11.1 Accuracy

Accuracy measures the overall correctness of the intent classification model by calculating the proportion of correctly predicted user query intents among the total number of evaluation samples. While accuracy gives a general performance overview, it serves as an initial benchmark to ensure that conversational queries in both English and Nepali are routed to their correct functional modules.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (3.2)$$

3.11.2 Precision

Precision indicates the proportion of user queries classified under a specific intent—such as a critical Safety Query—that are actually true instances of that intent. High precision is absolutely vital in a disaster response scenario to minimize false classifications of ordinary

informational queries as emergency requests, thereby ensuring that the processing backend remains clean and undistracted.

$$Precision = \frac{TP}{TP + FP} \quad (3.3)$$

3.11.3 Recall

Recall (Sensitivity) measures the classification model's inherent ability to correctly identify true safety or emergency instances from all actual valid samples present within the testing corpus. A higher recall ensures that fewer life-critical user questions escape proper routing, which is essential for preserving safety information integrity when citizens need immediate help.

$$Recall = \frac{TP}{TP + FN} \quad (3.4)$$

3.11.4 F1-Score

F1-score represents the mathematically balanced harmonic mean of precision and recall. It provides a robust evaluation metric that is especially useful when dealing with uneven class sentence distributions or dataset variations between standard English phrases and complex Nepali Devanagari Unicode inputs.

$$F1\text{-score} = \frac{2 \times (Precision \times Recall)}{Precision + Recall} \quad (3.5)$$

3.11.5 Confidence Threshold (C_t)

The Confidence Threshold (C_t) is a critical operational boundary parameter set to 0.7 (70%) within the bilingual NLP.js classifier. It serves as a quality gateway to control prediction uncertainty. When a user inputs a query, the model calculates a soft probability score $P(I_k|X)$ for each potential intent I_k given the input text X . If the maximum predicted probability fails to meet or exceed C_t , the classification routing forces a fallback routine.

$$Final_Intent = \begin{cases} \arg \max_k P(I_k|X), & \text{if } \max_k P(I_k|X) \geq 0.7 \\ \text{"Unknown / Fallback"}, & \text{if } \max_k P(I_k|X) < 0.7 \end{cases} \quad (3.6)$$

Setting $C_t = 0.7$ mathematically balances the system by filtering out ambiguous or noisy data translations while successfully processing clear multilingual requests during a high-stress disaster environment.

CHAPTER 4

METHODOLOGY

4.1 System Block Diagram

The proposed QSAFE Nepal earthquake response system is structured into three major components: the Input Section, Processing Section, and Output Section. This three-tier architecture enables the system to transform raw user queries and damage reports into verified, actionable safety responses and emergency information. The modular design ensures that each component can operate independently while maintaining seamless integration across the entire system.

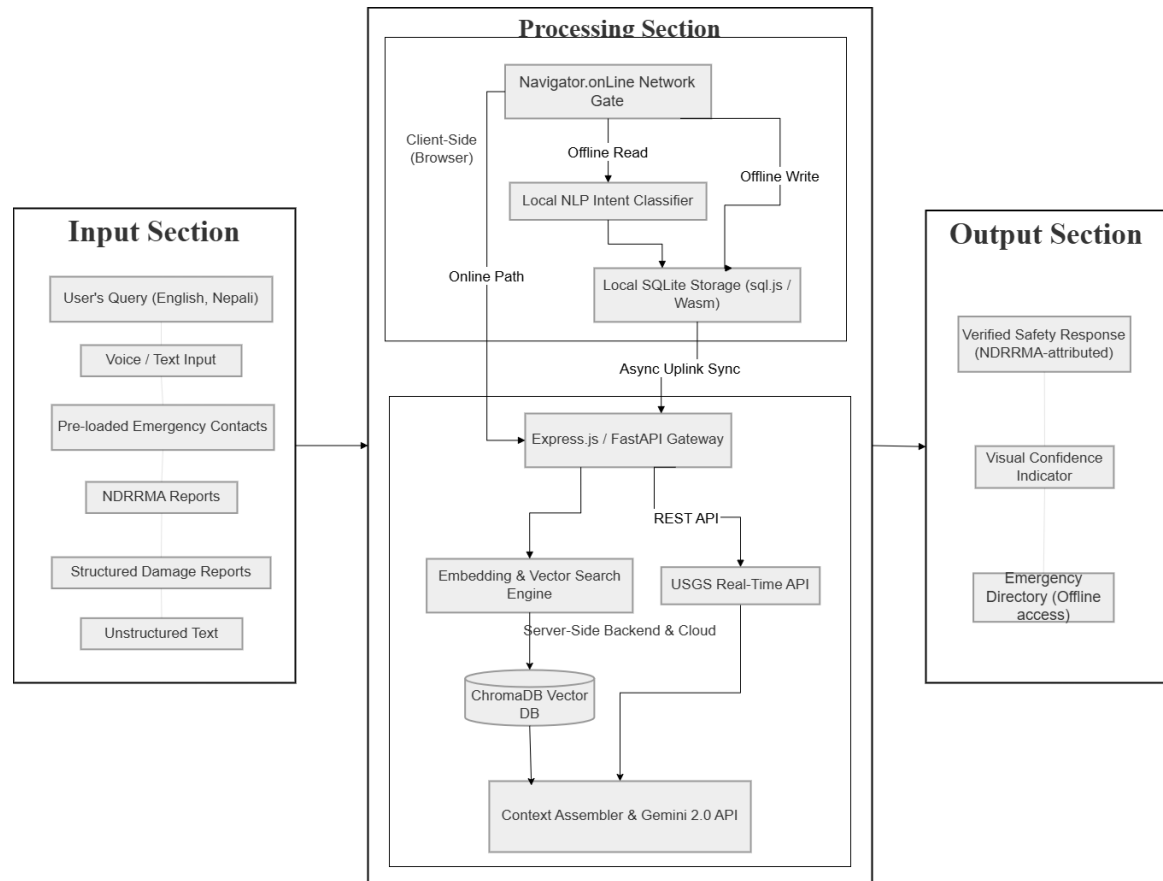


Figure 4.1: System Block Diagram of the Proposed multilingual earthquake management system with Explainability

4.1.1 Input Section

The Input Section is the entry point of the system, responsible for collecting, organizing, and preparing raw data that serves as input to the downstream processing components.

- **Multilingual Query Input:** Accepts textual queries in English or local scripts, handling both formal questions and fragmented, distressed input to ensure accessibility for all users. This flexibility allows the system to process ambiguous inputs common during emergencies, ensuring no critical safety query is missed regardless of the user's typing speed or stress level.
- **Structured Damage Report Form:** Captures essential damage data via a simplified interface: location (district or GPS), damage type (e.g., cracks, collapse), severity level (minor to severe), and optional descriptions, optimized for rapid mobile entry during emergencies. The form utilizes intelligent validation to guide users in providing precise details even when they lack technical expertise, ensuring the data collected is immediately usable for first responders.
- **Unstructured Text and Metadata:** Collects emergency reports, location names, and contextual details alongside automatic metadata such as submission timestamps and available location data to support accurate incident tracking. This dual-layer approach allows the system to index free-text reports for later analysis while simultaneously using metadata for real-time geospatial mapping of disaster zones.
- **Pre-loaded Emergency Contacts:** Embeds critical helpline numbers (Police, Ambulance, Disaster Management) and shelter locations directly into the app, ensuring instant access even when offline. By caching this data locally, the system guarantees that users in areas with destroyed cell towers can still locate help, effectively bridging the communication gap during the critical first hour of a disaster.

4.1.2 Processing Section

The Processing Section represents the core intelligence of the system, where data transformation, feature extraction, and classification are performed.

- **Network Gate Routing:** At the very front of the client application, the Navigator.onLine network gate checks the user's internet status. It acts as a traffic controller, instantly shifting user actions into either the direct online path, an offline read path, or an offline write path based on active network availability.
- **Local NLP and Storage:** When a device loses connection entirely, the offline read path routes queries to a local NLP intent classifier to detect languages and handle intent categorization directly on the device. Concurrently, any submitted forms run down the offline write path to be stored securely inside a local WebAssembly-backed SQLite

database (sql.js).

- **Background Syncing:** When a network connection becomes available again, the async uplink sync process triggers automatically. It reads any pending records stored within the local client database and pushes them up to the backend servers to clear the queue.
- **Backend API Gateway:** On the server side, an Express.js or FastAPI gateway serves as the primary ingestion point. It is responsible for handling incoming real-time traffic from connected clients as well as processing batch uploads coming from offline synchronization cycles.
- **Vector Search Indexing:** For incoming knowledge queries, the gateway sends text to the embedding and vector search engine. Here, text strings are converted into vectors to execute semantic searches against localized disaster management documents stored within the ChromaDB vector database.
- **External Telemetry Gathering:** In parallel to the text search pipeline, the backend utilizes a dedicated REST API client to monitor live seismic activity. This module regularly polls the USGS real-time API to pull the latest earthquake data and coordinates.
- **Context Assembly and Inference:** The final output is generated by a context assembler that merges the semantic document fragments retrieved from ChromaDB with the live telemetry from the USGS feed. This complete context bundle is sent directly to the Gemini 2.0 API, providing the language model with the exact facts needed to compile a reliable response.

4.1.3 Output Section

The Output Section presents the final results of the The Output Section details how the QSAFE Nepal system presents critical earthquake safety information to users in a clear, trustworthy, and actionable manner. The key components are structured as follows:

- **Verified Safety Response:** Outputs are derived exclusively from NDRRMA documents with explicit attribution (Document Section, Page, Topic) to enable independent verification and build user trust.
- **Visual Confidence Indicators:** Verified responses (≥ 0.70) display a green checkmark and the label “Verified Response”; unverified responses show a yellow warning and the label “Information Not Verified - Contact Authorities” for rapid reliability assessment.
- **Traceability and Transparency:** Responses include hyperlinks to source NDRRMA sections where available, clarifying that the system provides official government guidance rather than independent recommendations.
- **Damage Report Confirmation:** Users receive a confirmation message stating “Your report is saved locally. ID: [ID]. Syncs automatically when online,” with reports containing

unique IDs, timestamps, and location data for coordination.

- **Offline Emergency Directory:** Pre-cached, searchable access to critical resources including helplines (Police: 100, Ambulance: 102, NDRRMA: 16666), district agency contacts, and shelter locations, optimized for quick scanning under stress.

4.2 Dataset

The earthquake response and safety assistant system proposed in this research operates using a combination of authoritative reference materials, curated linguistic arrays, and dynamic client-side datastores. The data consists of localized safety protocols collected from official government disaster management portals and real-time user-reported incident telemetry, ensuring both administrative reliability and practical situational awareness. The data components used in this study serve explicit roles across intent classification training, semantic document retrieval, and disconnected local persistence. The QSAFE Nepal architecture utilizes three distinct data spaces:

1. **NDRRMA Earthquake Safety Manual** – The official, verified guideline document issued by the National Disaster Risk Reduction and Management Authority of Nepal [2]. This bilingual corpus contains localized procedures for structural reinforcement, immediate emergency actions, and post-seismic safety rules in both English and Nepali, serving as the primary knowledge base for the RAG pipeline.
2. **Localized Intent Dataset** – A custom-curated training collection containing over 200 semantic query variations distributed equally between English and native Nepali (Devanagari script) expressions [9]. This serves as the primary dataset for training and evaluating the NLP.js classification model.
3. **User-Generated Dynamic Dataset** – A client-side relational data structure captured locally via an offline-first SQLite WebAssembly compilation (`sql.js`) [10]. This dataset stores immediate field inputs—including system incident IDs, localized geolocation coordinates, damage severity classifications, and synchronization status flags—ensuring operational continuity during internet outages [11].

The combined data spaces cover multiple critical crisis domains such as seismic hazard instructions, civil safety checklists, live structural impact logs, and municipal emergency directories. This unified data approach prevents large language model hallucinations, guarantees cross-lingual language routing robustness, and significantly improves the platform’s reliability under extreme disaster conditions.

4.2.1 Nature of the Datasets

The system handles information across three distinct operational data spaces to guarantee factually grounded generation, text intent classification, and robust offline processing. Table 4.1 outlines the characteristics and classification metrics of these datasets, structured to align precisely within the document margins.

Table 4.1: Nature of Datasets Used

Dataset	Type	Language	Labels / Flags	Domain
NDRRMA Manual	Textual	English/Nepali	Section Metadata	Earthquake safety
Intent Dataset	Textual	English/Nepali	SAFETY / EMERGENCY	Query routing
Damage Reports	Relational	UTF-8 Text	0 = Pending, 1 = Synced	Incident logging

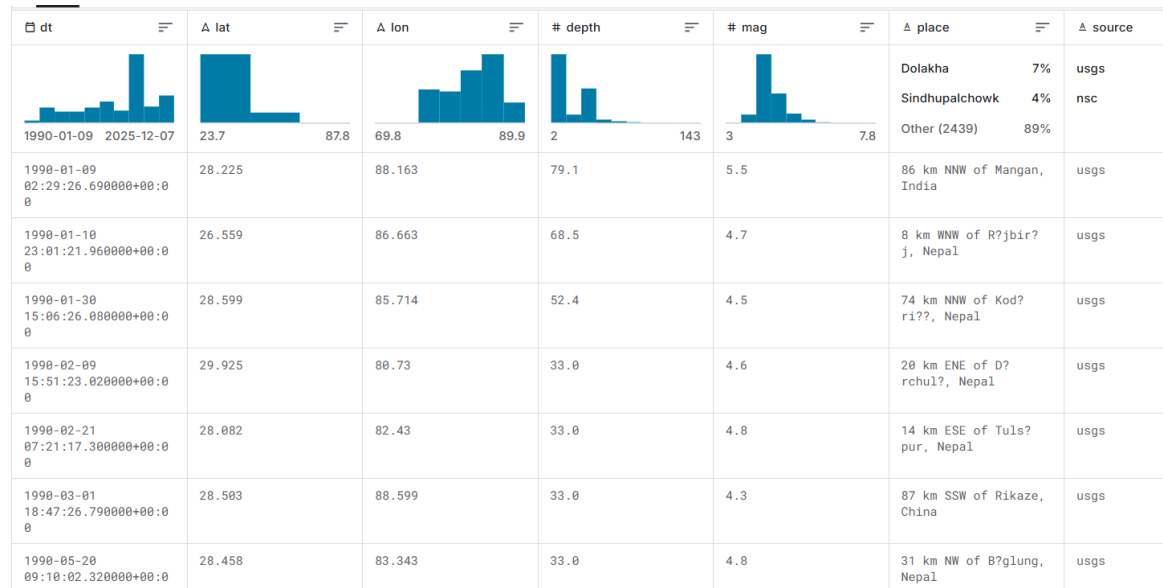


Figure 4.2: Nepal seismicity master.csv Dataset File [14]

4.2.2 Properties of the Datasets

(a) NDRRMA Safety Manual Dataset: This text corpus is split into discrete chunks to provide contextual reference text for the RAG pipeline. Table 4.2 defines its structural schema attributes.

Table 4.2: Properties of NDRRMA Manual Chunks

Attribute	Description
chunk_id	Unique identifier for the text segment
text_content	Raw emergency guide text string
chapter_ref	Origin chapter and section title

(b) Localized Intent Dataset: This training corpus contains user phrase variations paired with specific classification classes to train the NLP engine. Table 4.3 outlines its training properties.

Table 4.3: Properties of Intent Training Corpus

Attribute	Description
utterance	Natural language text phrase typed by the user
intent	Targeted classification route label

(c) User-Generated Dynamic Dataset (Local Database): This relational schema stores user-submitted reports directly on the client’s device when internet connection is lost. Table 4.4 details the field constraints for local client reporting logs.

Table 4.4: Properties of Local Client Damage Reports

Attribute	Description
report_id	Unique primary key for local records
coordinates	User device latitude and longitude
severity	Assigned hazard level (Low / Medium / High)
is_synced	Sync status flag (0 = Pending offline, 1 = Synced)

4.3 Preparation of Datasets

(a) Dataset Acquisition : The primary knowledge base comes directly from the official web portal of the National Disaster Risk Reduction and Management Authority (NDRRMA) of Nepal. This bilingual manual provides verified earthquake safety guidelines. To support this, we also gathered a custom dataset of 200 conversational phrases for intent classification and designed a relational database schema to log user reports.

(b) Data Cleaning : To ensure the text processes smoothly without system errors, we cleaned the raw data by applying a few essential steps:

- Extracting text from the official PDFs and stripping out irrelevant headers, footers, and extra whitespaces.
- Preserving full Unicode UTF-8 encoding so the native Nepali (Devanagari) script remains intact and uncorrupted.
- Removing duplicate phrases and empty entries from the conversational intent array.

(c) State and Label Standardization : We standardized our labels into clean, predictable formats to help route user requests accurately and manage data states behind the scenes:

- Intent Labels: User queries are mapped to clear operational categories like SAFETY_QUERY or EMERGENCY_REPORT inside the NLP engine.
- Sync Status Flags: User-submitted reports use a simple binary flag where 0 means pending offline and 1 means successfully synced to the cloud.

(d) Text Preprocessing and Ingestion : Before information can be searched or matched, it passes through a quick preparation pipeline:

- Overlapping Chunking: The clean manuals are broken into uniform, overlapping segments so vital context is not cut off at paragraph boundaries.
- Vector Embedding: These text segments are converted into dense vector coordinates using an embedding model, enabling rapid math-based similarity searches during a crisis.

(e) Dataset Partitioning : Instead of using a traditional machine learning train-test split, the data is partitioned functionally across the application's layers. The intent phrases train the conversational routing engine, the document chunks are indexed permanently in the vector database for retrieval, and the relational tables live directly on the client's device to capture local offline logs.

4.3.1 Dataset Statistics

To ensure balanced system performance across both languages, the static dataset elements are distributed uniformly across conversational intents and document retrieval chunks.

(a) Intent Dataset Distribution: Table 4.5 outlines how the conversational training expressions are split between English and native Nepali text to train the NLP routing engine.

Table 4.5: Class Distribution of Intent Dataset

Intent Class	Language	Approx. Count
Safety Query	English	50
Safety Query	Nepali	50
Emergency Report	English	50
Emergency Report	Nepali	50
Total	–	200

(b) NDRRMA Safety Manual Chunk Distribution: Table 4.6 shows the total volume of discrete, overlapping text segments generated from the official government publications and stored within the semantic vector database.

Table 4.6: Distribution of Knowledge Base Document Chunks

Document Source	Language	Approx. Chunks
Earthquake Safety Manual	English	120
Earthquake Safety Manual	Nepali	150
Total	–	270

4.4 Algorithm

The emergency assistance and query routing process follows a systematic algorithmic approach.

Algorithm Steps:

Step 1:

Start the user assistance and emergency routing process.

Step 2:

Accept the natural language text input or incident report provided by the user.

Step 3:

Preprocess the input string to remove text noise and normalize character sets across both supported language frameworks.

Step 4:

Pass the cleaned text to the natural language processing engine to classify the user's specific intent.

Step 5:

Evaluate whether the identified intent requires a real-time safety informational response or local emergency logging.

Step 6:

If the input is an emergency report, route the details to the client-side datastore to log the location coordinates and structural damage parameters directly on the device.

Step 7:

If the input is a safety query, query the localized vector repository to retrieve the most relevant reference text chunks from the safety manual guidelines.

Step 8:

Forward the retrieved safety context along with the original user query to the response generation layer.

Step 9:

Display the final verified safety answer or output the database storage synchronization status to the user.

Step 10:

End the user assistance and emergency routing process.

4.5 Flowchart

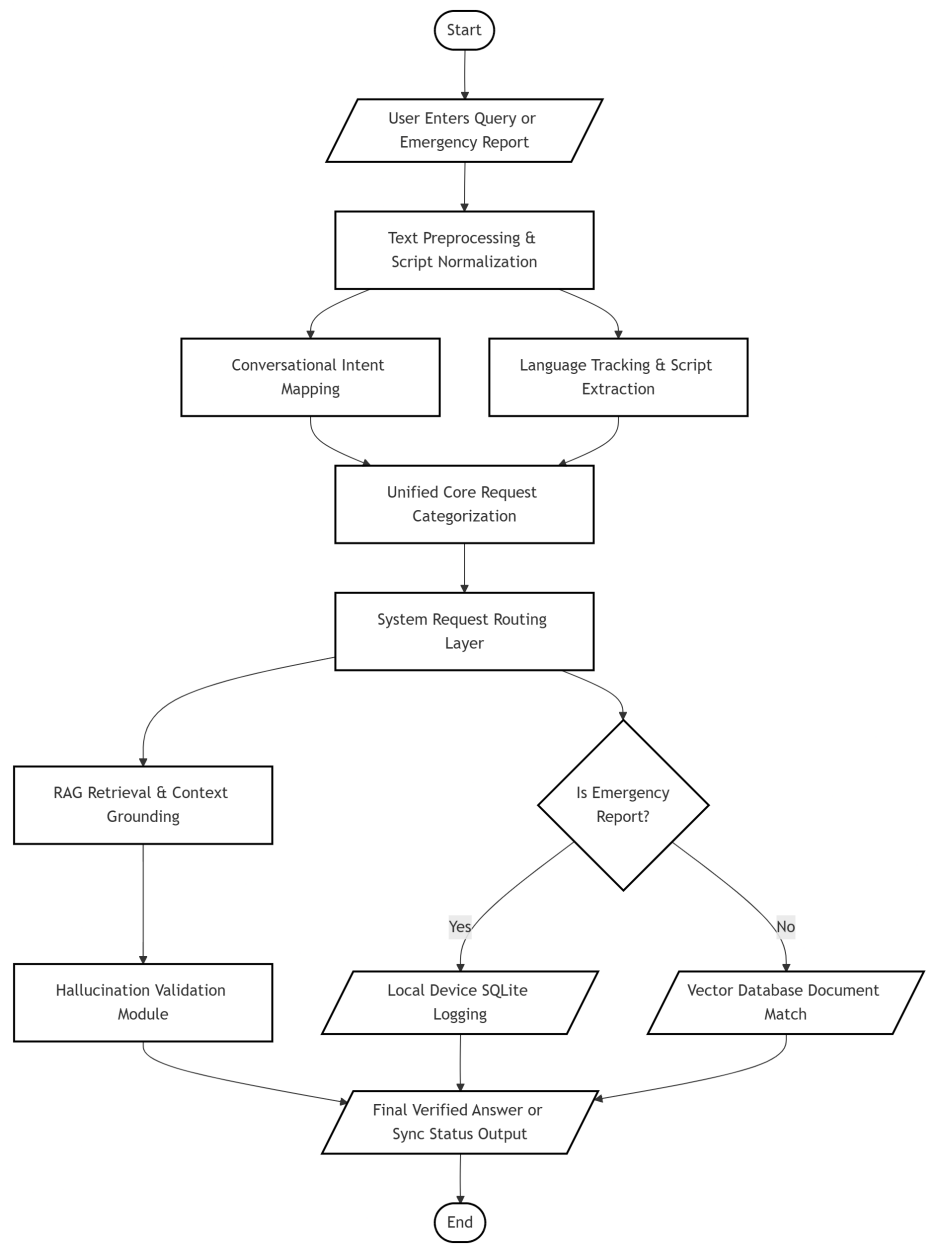


Figure 4.3: Flowchart of the A MULTILINGUAL EARTHQUAKE RESPONSE ASSISTANT with Explainability

4.6 Software Development Lifecycles

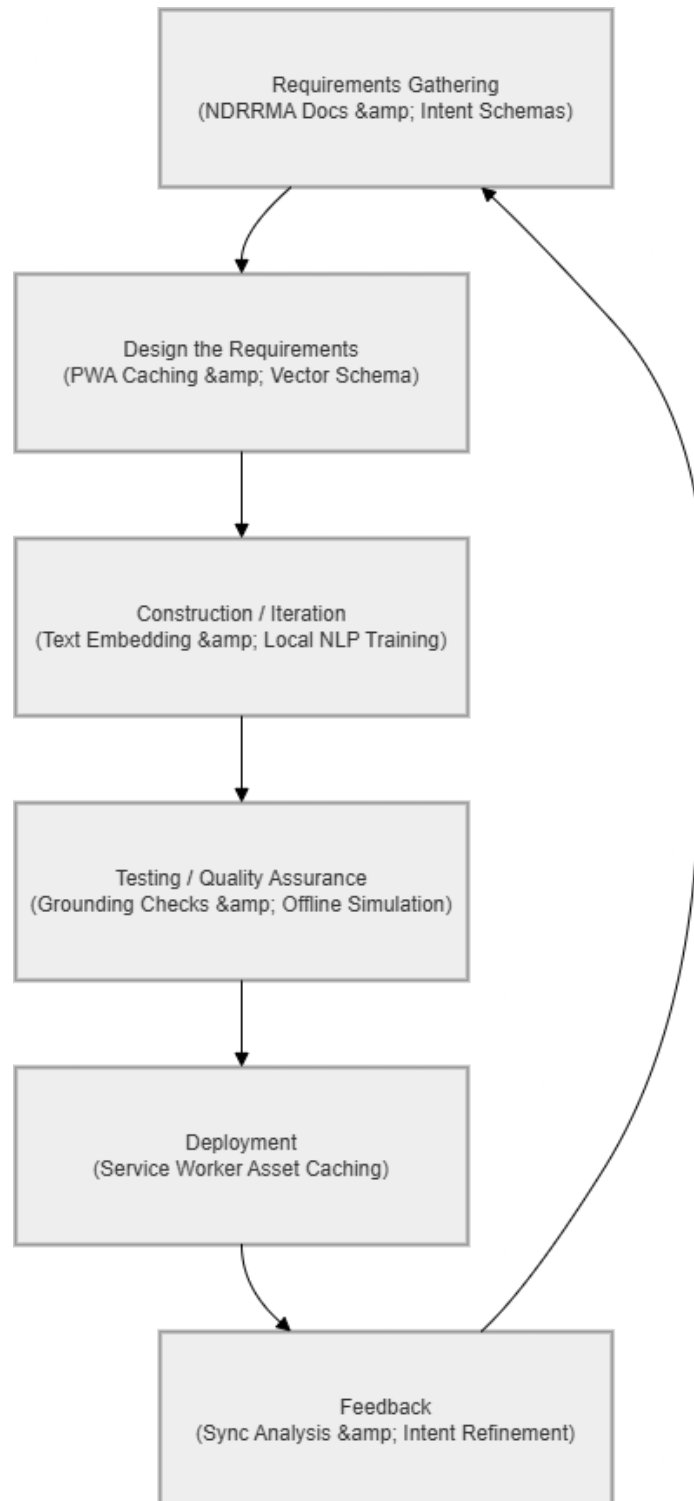


Figure 4.4: Agile Model Workflow with Feedback Loop

The development workflow illustrated in Figure 4.3 follows an Agile, iterative software development model. This approach emphasizes iterative development, flexibility, and

continuous feedback, making it suitable for retrieval-augmented generation (RAG) and offline-first assistant systems where knowledge base requirements and model performance evolve over time.

The process begins with Requirements Gathering, where the foundational objectives, official NDRRMA documentation assets, and localized bilingual intent schemas are defined. The system then transitions into Design the Requirements, which establishes technical parameters for PWA caching layers and server-side vector database schemas. The workflow moves directly into Construction / Iteration to handle practical system assembly, including text chunk embedding generation and client-side local NLP model training. Evaluation is carried out during the Testing / Quality Assurance phase, which subjects the application to rigorous grounding verification checks and comprehensive offline network isolation simulations. Upon successful validation, the system enters Deployment to execute critical service worker asset caching on the client's device hardware. Finally, the Feedback phase performs post-deployment synchronization recovery analysis and continuous linguistic intent refinement. If the system requires further optimization, this feedback is immediately incorporated, routing back to the beginning to repeat the development cycle. This iterative, feedback-driven process reflects the core principles of Agile development, ensuring adaptability, incremental progress, and continuous refinement.

4.7 Entity–Relationship (ER) Diagram

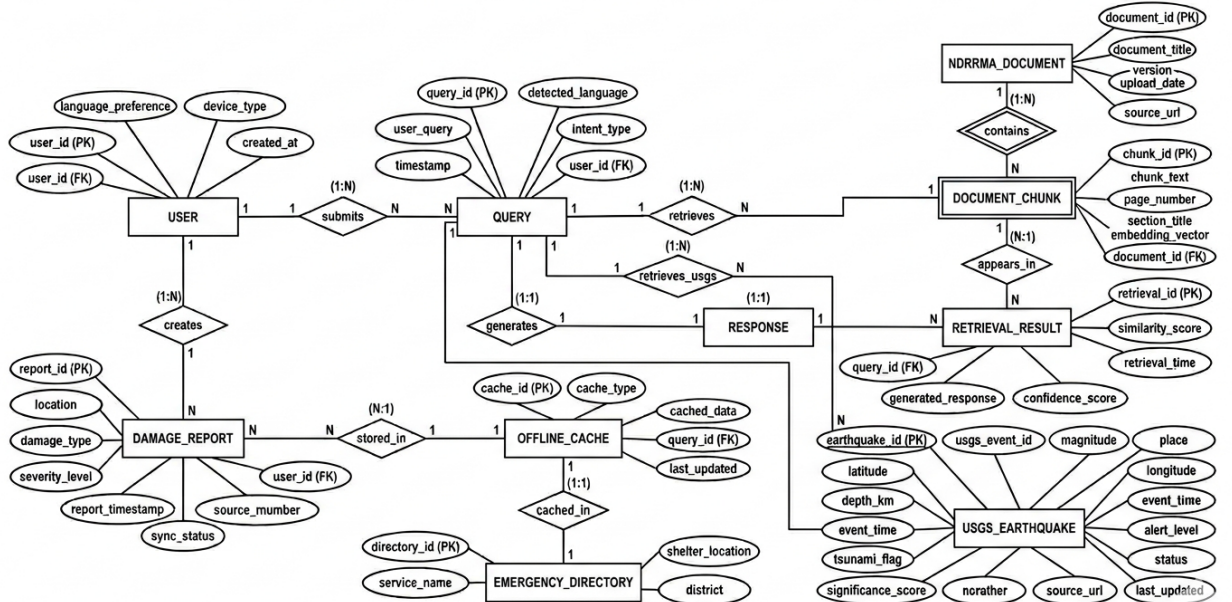


Figure 4.5: Entity–Relationship (ER) Diagram Of QSAFE Nepal

The Entity–Relationship (ER) diagram represents the overall data flow and structural design of the proposed earthquake response assistant system, encompassing user interaction, offline resilience, semantic document retrieval, and live telemetry validation. The system initiates with the `USER` entity, which stores essential user-related session details such as the user identifier, language preference, device type, and creation timestamp, representing individuals who interact with the assistant application. Users submit conversational requests or situational updates captured by the `QUERY` entity—containing attributes like the user query, timestamp, detected language, and intent type—and the `DAMAGE_REPORT` entity, which tracks localized impact metrics including location, damage type, severity level, report timestamp, and network synchronization status.

Each local interaction or reported incident undergoes a client-side management layer represented by the `OFFLINE_CACHE` entity. This stage manages local storage parameters, storing cache identifiers, cache types, raw cached text data, and updating intervals to maintain clean, transactional logging suitable for offline operations during infrastructure failures. The cached interaction boundaries are directly integrated with structural fallback resources inside the `EMERGENCY_DIRECTORY` entity, which contains pre-loaded safety fields including regional service names, emergency shelter locations, and specific districts to maintain localized operational capability without a network link.

The `NDRRMA_DOCUMENT` entity maintains structural metadata related to verified knowledge assets used for semantic grounding, tracking document titles, version details, official upload dates, and corresponding source URLs. Based on these verified source files, text data is divided into granular, tokenized components within the `DOCUMENT_CHUNK` entity, which holds raw chunk text, page numbers, section titles, and deep feature embedding vectors. When requests are queried, semantic similarities are mapped inside the `RETRIEVAL_RESULT` entity to record similarity scores and retrieval times, while real-time environmental events are actively streamed into the `USGS_EARTHQUAKE` entity to log live seismic data such as event identifiers, magnitude scales, geographic places, coordinates, depth parameters, and tsunami alert flags.

To address the hallucination and reliability limitations of standard generative models, the architecture executes a grounded pipeline culminating in the `RESPONSE` entity. This entity maps directly back to the active user requests, recording the final generated response text alongside an associated confidence score that reflects the absolute grounding accuracy and algorithmic reliability of the output. Overall, the ER diagram illustrates a structured and transparent data pipeline that integrates decentralized local client caching, vector-driven retrieval-augmented generation (RAG), real-time API tracking, and bilingual context anchoring for effective, high-fidelity earthquake response assistance.

4.8 Use Case Diagram

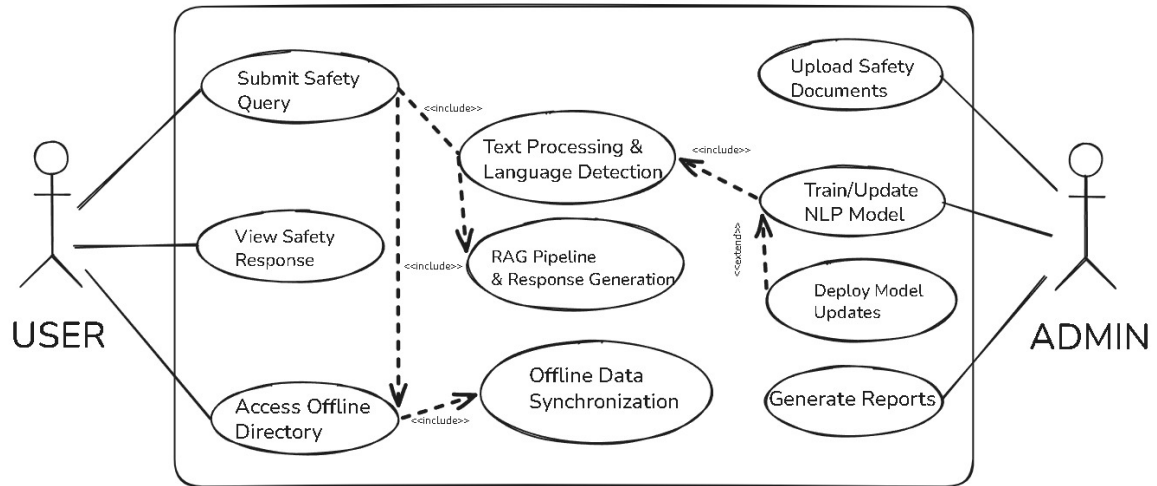


Figure 4.6: Use Case Diagram Of QSAFE Nepal

The use case diagram shown represents the interactions between the user and the QSAFE Nepal earthquake response assistant system. It illustrates how users submit emergency queries or local damage reports to find verified disaster protocols and receive reliable, context-grounded guidance generated using an offline-first Retrieval-Augmented Generation (RAG) architecture. The diagram highlights essential system functionalities, including bilingual query handling, local intent classification, offline contact directory access, and client-side database logging.

Additionally, the diagram includes an administrative actor responsible for managing official NDRRMA knowledge bases, training and updating the localized NLP model, and deploying critical service updates. Only key `<include>` and `<extend>` relationships are used to maintain clarity, where mandatory processes such as text preprocessing and language tracking are included within core use cases, and optional features such as asynchronous data synchronization extend the baseline local logging functionality when network integrity is restored. This representation provides a clear and structured overview of the system's functional behavior from the user's perspective.

CHAPTER 5

EXPECTED OUTPUT

The proposed project aims to develop an effective and resilient emergency assistance and query routing system using localized natural language processing and retrieval architectures. By leveraging the combined strengths of an NLP intent classification engine and a Retrieval-Augmented Generation (RAG) framework, the system is expected to accurately route emergency reports while also providing verified safety guidelines. The integration of an offline-first database layout addresses the limitations of traditional cloud-dependent applications and enhances system reliability and public trust during critical network infrastructure failures.

The expected outcomes of the proposed system are as follows:

- The system shall accurately classify and route user requests into informational or emergency tracks using a localized NLP intent-mapping architecture.
- The system shall seamlessly process and normalize text inputs across both English and native Devanagari scripts while preserving complete Unicode data integrity.
- The system shall provide reliable local data logging by serializing location coordinates and incident parameters directly into an offline, client-side database.
- The system shall present grounded, hallucination-free responses to safety queries by extracting contextual reference text from localized NDRRMA manuals.
- The system shall achieve effective performance in request routing and information retrieval, evaluated using standard metrics such as classification accuracy, query latency, and database synchronization success rates.
- The system shall offer a user-friendly interface for inputting emergency queries and visualizing safety guidelines alongside local report synchronization statuses.

BIBLIOGRAPHY

- [1] National Disaster Risk Reduction and Management Authority (NDRRMA), “Nepal Disaster Risk Reduction Portal: 2015 Gorkha Earthquake Impact Summary,” Government of Nepal, Kathmandu, 2021. <https://www.ndrrma.gov.np>
- [2] National Disaster Risk Reduction and Management Authority (NDRRMA), “Earthquake Safety Manual,” Government of Nepal, Kathmandu, 2020. <https://www.ndrrma.gov.np>
- [3] P. Meier, *Digital Humanitarians: How Big Data Is Changing the Face of Humanitarian Response*. CRC Press, Boca Raton, FL, 2015.
- [4] Central Bureau of Statistics Nepal, “National Population and Housing Census 2021: National Report,” Government of Nepal, Kathmandu, 2021. <https://censusnepal.cbs.gov.np>
- [5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [6] A. Ranganathan, M. Yenduri, and H. Ramamurthy, “Disaster response chatbots: Capabilities and limitations,” *International Journal of Disaster Risk Reduction*, vol. 65, p. 102538, 2021.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, “The state and fate of linguistic diversity and inclusion in the NLP world,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 6282–6293, 2020.
- [9] AgoraIO and Contributors, “NLP.js: Natural Language Processing in Node.js,” GitHub repository, 2020. <https://github.com/axa-group/nlp.js>
- [10] sql.js Contributors, “sql.js: SQLite compiled to WebAssembly,” GitHub repository, 2021. <https://github.com/sql-js/sql.js>
- [11] M. Gaunt, “Service Workers: An Introduction,” Google Developers Web Fundamentals, 2019. <https://developers.google.com/web/fundamentals/primers/service-workers>

- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [13] L. Ammann, S. Ott, C. Landolt, and M. Lehmann, “Securing RAG: A Risk Assessment and Mitigation Framework,” *arXiv preprint arXiv:2505.08728*, May 2025. <https://doi.org/10.48550/arXiv.2505.08728>
- [14] A. Singh, “Nepal Earthquake Seismicity Dataset (1990 - 2026),” *Kaggle*, 2026. <https://www.kaggle.com/dsv/14407708>
- [15] A. Zanini, “Understanding Vector Databases: The Engine Behind Modern AI,” *Bright Data Blog*, 2026. <https://brightdata.com/blog/ai/vector-databases>

CHAPTER A

APPENDIX

A.1 Gantt Chart / Project Timeline

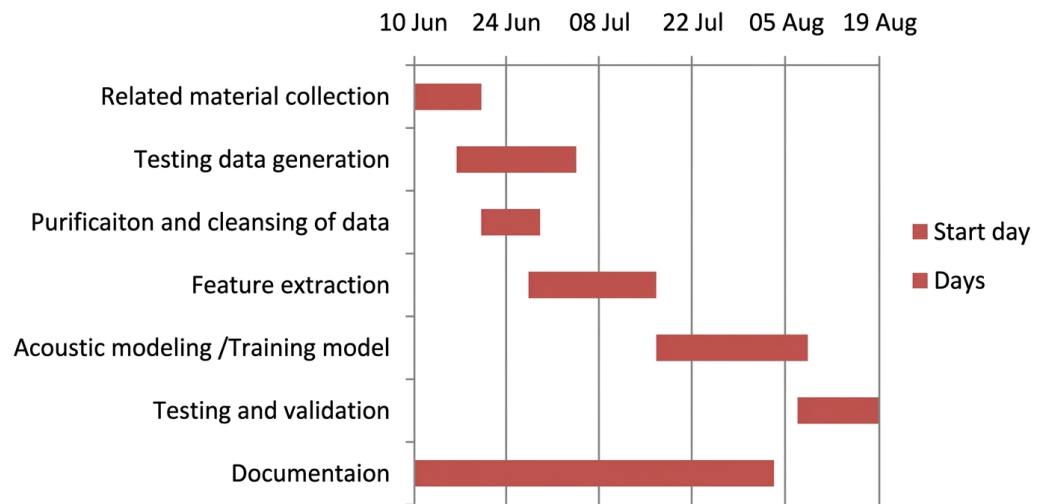


Figure A.1: Gantt Chart for QSAFE Nepal. Project Timeline (June–August 2026)